

企微 AI 机器人架构：一个人和一个智能体

从 HTTPS 回调到异步处理、知识库、问答、日报和好友自动通过，拆开一套真实运行系统的结构与取舍。

← 返回运营栏目

运行 53 天

消息留档 17,488 条

智能体处理 12,846 条

好友自动通过 265 人

运营数据看板 →

01 · OVERVIEW

全景：一条回调流水线，加一排定时任务

核心思路一句话：**企微的每一条消息都会以 HTTPS 回调的形式打到我的服务器**，一个常驻进程负责收下、留档、分发；智能问答、日报、加好友都是这条流水线上的分支。



旁路全部是 systemd timer 拉起的一次性脚本，和常驻进程共享同一份留档与状态文件。问答不经过 Claude Code——是进程内直接调 LLM API，这样单条延迟稳定在秒级。

02 · PRINCIPLES

先立规矩，再写代码

这套系统迭代了两个多月，功能一直在加，但底下这五条从第一天定下后没变过——它们比任何一段代码都值得抄。

P1

纯标准库，零框架

整个系统是 Python 标准库 + SQLite，**没有 Web 框架、没有消息队列、没有构建步骤。**一台最小配置的云主机就能跑。规模没到之前，每引入一个依赖都是在给未来的自己埋运维债。

P2

先留档，再谈功能

每条回调先原样落盘再处理。这份日志后来长出了**上下文理解、群日报、运营看板、知识库**四个功能——当时一个都没想到。原始数据是资产，功能是利息。

P3

该松则松，该紧则紧

功能性环节全部 **fail-open**：知识库挂了、联网挂了，问答降级照常跑。涉及敏感话题、对外承诺的环节全部 **fail-close**：判定字段缺失就当命中，宁可不说。

P4

状态即文件

去重库、对话线程、日报水位，全是 SQLite 单文件和水位小文件。**迁移服务器 = 把文件拷走。**换来的代价是这些文件必须当资产管——丢了就会重复发日报、重复加好友。

P5

人机边界画清楚

本人手动回复某个客户后，机器人对**这个客户**自动闭嘴一段时间（滑动窗口）；知识库涉及报价、服务承诺的条目默认不放行——**机器人可以答题，不可以替我许诺。**

P6

监控独立于被监控者

LLM 链路每 5 分钟被探测一次，连续失败就**私聊我本人告警**——告警消息走企微通道，不经过被监控的 LLM 链路。用会坏的东西报告它自己坏了，等于没有监控。

群问答的闸门链：大部分消息的正确处理是「闭嘴」

群里 90% 的消息不该机器人接话。所以问答链路的主体不是「怎么答」，而是一串**决定不答**的闸门——消息要连过七道闸，才轮得到生成答案。



橙色 = 拦截型闸门 (fail-close)，蓝色 = LLM 调用。G2 与 G7 之间的所有判定共享 G2 那一次调用的输出，全链路只有两次大模型调用。

关键决策 · 语气

像我，而不是像客服

回答的说话风格不是拍脑袋写 prompt，是拿**我本人在群里的真实发言**校准出来的：逗号多、句号少、允许口头语。改风格后必须跑**固定用例的实测探针**——读 prompt 觉得没问题和真的没问题，是两回事。

关键决策 · 追问

机器人听不见自己

回调平台不回显机器人经 API 发出的消息，于是「你是 Win 还是 Mac？」——「我是win」这种接话，分诊器根本不知道在回什么。解法是**把机器人的上一条答案存下来，喂回给分诊器**，再配上限次数的对话窗口，聊得起来也收得住。

关键决策 · 防雷同

同一个人，别复读

同人短时间连续提问时，把机器人最近几条旧回答喂回去，明确要求「换角度、更深入、别复读」。防复读不靠温度参数，靠**让模型看见自己说过什么**。

六个模块，共用一条数据底座

每个模块都是独立脚本，靠同一份留档日志和几个 SQLite 文件协作——没有模块间调用，坏一个不连坐。

群日报

每天 05:00

取近 24 小时群消息，LLM 归纳出话题、金句、求助与回应，渲染成**HTML 长图**发回各群。
水位文件记住上次截止点，服务重启不会重发。

好友自动通过

事件触发

好友申请事件 → 自动通过 → **自动邀请进群** → **发欢迎语**，一气呵成。
去重库保证一个人只处理一次，失败会解除标记允许重试。

本地知识库

每天 05:30 增量

两个来源：我在企微**手敲的回复**（LLM 蒸馏成问答对）和**博客文章**（每篇提炼要点、带期号与链接）。答题前检索注入，机器人先说「我的口径」，答不到再谈通识。

能力画像问卷

独立服务

群友能力问卷，微信扫码/一键登录，答完生成能力徽章。反注水靠「多轴证据 ≥ 2 根支撑」——单轴吹得再高也折价。

链路健康监控

每 5 分钟

定时探测 LLM 链路，**连续 3 次失败才告警**（防抖动误报），恢复后再报一次平安。
告警通道与被监控链路完全独立。

彩虹屁语音

关键词触发

群里说「夸夸」，LLM 根据那个人的近期发言写一段针对性夸赞，用**我的克隆音**合成语音发进群。纯属好玩，但它是群里传播度最高的功能。

踩过的坑，比架构图值钱

下面这些结论每一条都对应一次真实故障或一次返工。如果你只带走一部分内容，带走这段。

01 别把判定押在平台的字段格式上

回调平台改过一次消息类型的编码方式，凡是按 `msgType` 精确值判断「这条是不是我本人发的」的逻辑当场全崩。后来一律改成**恒等式判定**：发送者 ID 等于账号主 ID 就是本人。字段格式是别人的，恒等关系才是你的。

02 让 LLM 输出中文长文时，别用 JSON

结构化输出里一旦含中文长正文（引号密集），模型抄写时不转义，JSON 解析大面积失败——实测三篇坏两篇。改成**逐行键值的行格式**后零失败，还有个额外好处：输出被截断时前面的字段照样能用，JSON 则整条作废。

03 知识的时效性没法在入库时解决

「当时没问题」的回答不能当「现在没问题」讲。正解是**每条知识带日期入库**，分成长期有效 / 时段信息 / 易变信息三档，检索注入时把日期一起给模型、让它自己把时间差说清楚——「这是我上个月验证过的情况，建议以最新实测为准」。

04 「敏感」不只是泄密，还有替你许诺

知识库敏感清单第一版只盯着报价、渠道这类「泄露信息」，漏了一整类：**机器人替我做出的服务承诺**。它说出口，兑现压力都在我身上。这类条目默认不放行，放行必须人工逐条来。

05 改判定类 prompt，必须跑对照组

LLM 判定本身有噪声：同一版 prompt 跑两遍，边缘字段也会有出入。不先测**自一致性基线**就对比新旧版本，会把模型噪声当成你改出来的回归——或者反过来，把回归当噪声放过去。

06 验证从用户视角发起，别信中间状态

「服务日志说发送成功」和「群里真的收到了」之间隔着好几层。每个功能上线的最后一步都是**真实端到端**：真群发一条、真手机看一眼。同理，「配置看着对」不等于「运行时在用这份配置」——验证要在真实运行时做。

07 凡是「花钱或有副作用」的调用，失败了不许盲目重试

发消息、加好友、买资源这类操作，「结果未知」时重试可能造成重复动作。先查副作用有没有发生（按 ID 做差集），确认没发生再重试。**幂等性不是库给你的，是你设计出来的。**

用了什么，各自管什么

运行时

Python 标准库

无框架无依赖（仅语音链路引入
ffmpeg / pilk）

数据

SQLite + 追加日志

去重 / 线程 /
知识库各一个单文件库

调度

systemd service + timer

常驻 2 个服务，定时 4 个 timer，
独立系统用户运行

入口

Caddy

TLS 自动证书，
请求只进本机回环端口

企微接入

第三方托管接口

负责收消息回调、调发送接口，
个人主体可接入

分诊 / 解析

Claude Sonnet

一次调用输出全部判定字段，
省钱且一致

作答

Claude Opus

口径与语气的主力，
值得用最好的模型

视觉 / 联网

Gemini Flash • Sonar

群图转文字描述；
时效性问题联网检索

服务器怎么配：一台小主机，跑成系统服务

没有集群、没有容器编排。一台最小规格的云主机就够（2 核 2G，**不需要 GPU**——
模型全在云端 API），每个模块跑成 systemd 服务。这一层越「无聊」，系统越稳。

R1

独立低权限用户

服务跑在专用系统用户下，**不用 root**，
也不和机器上其他业务共用身份。
文件权限跟着收紧：密钥文件只有属主可读，
数据文件组外不可见。

R2

密钥只活在一个文件里

所有密钥集中在一份 .env（chmod 600），
代码零硬编码，仓库里只有留空的 example。
踩过的坑：「**谁来读这份文件**」
比文件本身更重要——每个 unit 都要显式声明
EnvironmentFile，
少一个就会静默落回代码默认值。

R3

三道安全底线

回调入口校验鉴权头，陌生请求一律 200
空应答不暴露信息；**发送动作有白名单兜底**——
就算上层逻辑写错，消息也发不到不该发的人；
服务只监听本机回环，对外只有 443。

```
# /etc/systemd/system/bot.service — 常驻服务的骨架，timer 同理
[Unit]
Description=bot receiver
After=network-online.target

[Service]
User=botuser # 独立低权限用户，不用 root
WorkingDirectory=/opt/bot/app
EnvironmentFile=/opt/bot/app/.env # 密钥只在这份文件里，chmod 600
ExecStart=/opt/bot/venv/bin/python receiver.py
Restart=always

[Install]
WantedBy=multi-user.target

# 定时模块各配一个 .timer: OnCalendar=*-*-* 05:00 + Persistent=true (停机错过自动补跑)
```

另一件比配置更重要的事：去重库、对话线程、日报水位这些**状态文件是资产**——迁移服务器时和 .env 一起带走；丢了它们，机器人会重复发日报、重复加好友。定期备份。

08 · KNOWLEDGE BASE

知识库怎么搭：让机器人先说「我的口径」

通用大模型答得了通识，答不了「你怎么看」。知识库解决的就是这件事——把我说过的话变成机器人的第一优先级。**没有向量库、没有 embedding 服务**：几百条的量级，一张 SQLite 表加关键词打分就够了。

源 1

我手敲的回复

从留档日志重建会话，抽出「客户问 + 我答」的段落，LLM 逐段蒸馏成问答对——寒暄、约时间、私事直接丢弃（丢弃率约六成，正常）。
机器人经接口发的消息不会出现在留档里，所以「这条是我本人说的」这个判据天然干净。

源 2

我的博客文章

按站点地图逐篇抓正文，每篇蒸馏几条要点，**带期号和原文链接**入库。机器人答题时能说出「我第 X 期专门讲过，链接在这」——回答本身就在引流。

```
-- 一张表就是全部 (SQLite, 字段示意, 按你的场景增删)
knowledge(
  question TEXT,      -- 归一后的问法
  answer   TEXT,      -- 以「我」的口径给出的答案
  keywords TEXT,      -- 检索用关键词
  said_on  TEXT,      -- 这句话是什么时候说的: 时效判断的根
  temporal TEXT,      -- evergreen / dated / volatile 三态
  sensitive INTEGER,  -- 默认 1=不放行, 人工审核后置 0
  source   TEXT       -- reply 手敲回复 / blog 博客蒸馏
)
```

检索

打分，不做向量

中文按 2-gram 重叠度对问法和关键词打分，取最相关的几条、低于阈值一条都不注入。
几百条量级上，向量库是过度工程——
先让检索跑起来，等库真的大了再升级。

注入

日期跟着知识走

每条带着日期进 prompt;
时点信息和易变信息**一律注明**
「这是当时的状态」，
让模型自己把时间差说清楚，
而不是把旧口径当成现状讲。

运维

增量 + 人工把关

每天一次定时增量（新回复 + 新文章）；
敏感条目默认不放行，放行必须人工逐条审。
整条链 fail-open: **知识库挂了，问答降级照常跑。**

把这页丢给你的 Claude Code

这套系统就是我和 Claude Code 一起从零写出来的——你也可以。但别让它照抄这页：我的取舍长在我的场景上。让它先问清你的场景，再设计你的版本。

起手可以这样聊（按顺序）

1. **定范围**：「我有 N 个群 / 主要是私聊咨询，我想让机器人做什么、绝对不做什么？」——先写下不做什么。
2. **选接入**：「帮我调研个人可用的企微/微信消息回调方案，对比合规风险和稳定性。」
3. **搭底座**：「先只做两件事：收回调全量落盘 + 一个能手动触发的回显。跑通再谈智能。」
4. **加闸门**：「参考这页的闸门链（先判『该不该答』再谈『怎么答』），按我的群设计闸门顺序。」
5. **立红线**：「哪些话题固定话术拒答？机器人能不能谈价格、做承诺？人工接管怎么触发？」
6. **再谈锦上添花**：日报、知识库、语音，都是流水线跑稳之后的分支。

※ 这一页刻意省略了运维细节、凭据管理和完整配置——不是藏私，是这些东西照抄必坏：它们和服务器环境、账号形态、业务边界强耦合。把「思路」拿走，让你的 CC 长出「实现」。

10 · FURTHER READING

扩展阅读

企微协议接入 · 开发文档

我用的第三方托管平台：回调事件与发送接口的参考

Claude API 文档

分诊与作答用的模型；Messages API 与提示词设计

OpenRouter

多模型聚合网关：一个接口调 Claude / Gemini / Sonar

Perplexity Sonar

联网检索模型：给答案补时效的那一环

systemd.timer

定时任务不用 cron 用 timer：日志、依赖、防重叠都省心

SQLite

单文件数据库：这套系统全部状态的家

Caddy

自动 HTTPS 的 Web 服务器，配置只有几行

xntj.tv · 张拼拼的博客

73 期实战记录——知识库的第二个来源就是它

私域运营数据看板

这套系统跑出来的真实数据：消息、分层、线索与商机

关于这页 · 分享性质，非教程；架构与数据均为真实线上系统（截至 2026-08-04），凭据、服务器信息与完整配置已刻意省略。文中「CC」指 Claude Code。想聊这套系统，来博客 xntj.tv 找我，或让把这页发给你的人拉你进群。